

Validación de Unicidad en JPA con Spring Boot y Jakarta EE

Para garantizar la *unicidad* de un campo o un conjunto de campos en una entidad JPA con **Spring Boot** y **Jakarta EE**, tienes varias opciones, aunque las anotaciones de validación estándar no incluyen una verificación directa para unicidad. A continuación te explico cómo se puede manejar la unicidad a nivel de JPA y validación.

1. Unicidad a nivel de base de datos: Anotación `@UniqueConstraint` (JPA)

Para garantizar que un campo o un conjunto de campos sea único a nivel de base de datos, se utiliza la anotación `@UniqueConstraint` en la clase de la entidad JPA. Esto se asegura de que la base de datos aplique una restricción de unicidad en la columna o combinación de columnas especificadas.

Ejemplo: Unicidad de una columna (`email`):

```
@Entity
@Table(name = "usuarios", uniqueConstraints = {
    @UniqueConstraint(columnNames = "email")
})
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String email;

    // Otros campos y métodos
}
```

En este ejemplo, la columna `email` tendrá una restricción de unicidad, tanto a nivel de JPA como en la base de datos. La clave aquí es la combinación de `@Table` con `@UniqueConstraint`, que permite especificar una o más columnas únicas.

Ejemplo: Unicidad de un conjunto de columnas (`email` y `nombre`):

```
@Entity
@Table(name = "usuarios", uniqueConstraints = {
    @UniqueConstraint(columnNames = {"email", "nombre"})
})
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```

    @Column(nullable = false)
    private String email;

    @Column(nullable = false)
    private String nombre;

    // Otros campos y métodos
}

```

En este caso, la combinación de **email** y **nombre** debe ser única. No podrá haber dos filas en la base de datos con la misma combinación de valores para ambos campos.

2. Validación de unicidad a nivel de aplicación: Personalizando validaciones

Para validar la unicidad a nivel de aplicación antes de que los datos lleguen a la base de datos, puedes crear una **validación personalizada** con **Jakarta Bean Validation**.

Pasos para crear una validación personalizada de unicidad:

1. **Crear la anotación personalizada** para validar la unicidad de un campo o conjunto de campos.
2. **Crear el validador** que verifica si el valor ya existe en la base de datos.
3. **Usar la anotación en la entidad** para validar antes de que se inserten los datos.

Paso 1: Crear la anotación personalizada @UniqueValue

```

import jakarta.validation.Constraint;
import jakarta.validation.Payload;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Constraint(validatedBy = UniqueValueValidator.class)
@Target({ ElementType.FIELD })
@Retention(RetentionPolicy.RUNTIME)
public @interface UniqueValue {
    String message() default "El valor debe ser único";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
    Class<?> entity(); // Entidad a validar
    String fieldName(); // Nombre del campo en la entidad
}

```

Paso 2: Crear el validador UniqueValueValidator

El validador verifica en la base de datos si el valor del campo ya existe. Utiliza el **EntityManager** de JPA para consultar la existencia del valor.

```
import jakarta.validation.ConstraintValidator;
import jakarta.validation.ConstraintValidatorContext;
import org.springframework.beans.factory.annotation.Autowired;

import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;
import javax.persistence.Query;

public class UniqueValueValidator implements ConstraintValidator<UniqueValue, Object> {

    private String fieldName;
    private Class<?> entityClass;

    @PersistenceContext
    private EntityManager entityManager;

    @Override
    public void initialize(UniqueValue constraintAnnotation) {
        this.fieldName = constraintAnnotation.fieldName();
        this.entityClass = constraintAnnotation.entity();
    }

    @Override
    public boolean isValid(Object value, ConstraintValidatorContext context) {
        if (value == null) {
            return true; // Valor nulo lo manejamos con @NotNull
        }

        Query query = entityManager.createQuery(
            "SELECT COUNT(e) FROM " + entityClass.getName() + " e WHERE e." +
        );
        query.setParameter("value", value);

        Long count = (Long) query.getSingleResult();
        return count == 0; // Retorna verdadero si no existe, es decir, es único
    }
}
```

Paso 3: Usar la validación en la entidad

Aplicamos la anotación **@UniqueValue** en la entidad donde queremos garantizar la unicidad. En este caso, validamos que el **email** sea único en la entidad **Usuario**.

```
@Entity
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @UniqueValue(entity = Usuario.class, fieldName = "email", message = "El email
    @Column(nullable = false)
    private String email;

    // Otros campos y métodos
}
```

3. Combinación de validación personalizada y restricción de base de datos

Aunque puedes implementar la validación a nivel de aplicación, **la mejor práctica es siempre combinarla con la validación a nivel de base de datos** utilizando el enfoque de **@UniqueConstraint** en la tabla JPA, ya que la base de datos es el último nivel de control de los datos. Esto asegura que incluso si por alguna razón no se valida en la aplicación, la base de datos mantendrá la integridad.

Conclusión:

Para garantizar la unicidad de campos en tu aplicación de **Spring Boot** con **Jakarta EE**, debes combinar validaciones a nivel de base de datos mediante **@UniqueConstraint** con validaciones a nivel de aplicación cuando sea necesario mediante un validador personalizado. Esto asegura que el dato sea verificado tanto en la aplicación como en la base de datos, evitando duplicados no deseados.